

Offline 2D-to-3D Reconstruction System for ARM-Based Mobile Devices

PROJECT REPORT

Submitted by

Goureesh Chandra (TVE22CS069)

Ivin Mathew Kurian (TVE22CS075)

Muhammed Farhan (TVE22CS094)

Rethin Francis (LTV22CS149)

to

The APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree

of

Bachelor of Technology

in

Computer Science and Engineering



Department of Computer Science and Engineering

College of Engineering Trivandrum

Kerala

March 21, 2026

DECLARATION

We undersigned hereby declare that the project report **Offline 2D-to-3D Reconstruction System for ARM-Based Mobile Devices**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of **Prof. Divya S K**. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Thiruvananthapuram

Date: March 21, 2026

Goureesh Chandra
Ivin Mathew Kurian
Muhammed Farhan
Rethin Francis

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING TRIVANDRUM**



CERTIFICATE

This is to certify that the report entitled "**Offline 2D-to-3D Reconstruction System for ARM-Based Mobile Devices**", submitted by **Goureesch Chandra** (TVE22CS069), **Ivin Mathew Kurian** (TVE22CS075), **Muhammed Farhan** (TVE22CS094), **Rethin Francis** (LTV22CS149) to the **APJ Abdul Kalam Technological University** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Prof. Divya S K

Assistant Professor

Department of CSE

(Guide)

Prof. Rani Koshy

Associate Professor

Department of CSE

(Coordinator)

Dr. Ajeesh Ramanujan

Associate Professor

Department of CSE

(Head of Department)

Signature of Examiner:

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude and heartfelt indebtedness to our guide **Prof. Divya S K**, Assistant Professor, Department of Computer Science and Engineering for his/her valuable guidance and encouragement in pursuing this project.

We are also thankful to **Dr. Ajeesh Ramanujan**, Head of the Department for his help and support. We also extend our hearty gratitude to project co-ordinators, **Prof. Rani Koshy**, Associate Professor, College of Engineering Trivandrum for providing necessary facilities and their sincere co-operation.

Our sincere thanks is extended to all the teachers of the Department of Computer Science and Engineering and to all my friends for their help and support.

Above all, we thank God for the immense grace and blessings at all stages of the project.

Goureesh Chandra (TVE22CS069)

Ivin Mathew Kurian (TVE22CS075)

Muhammed Farhan (TVE22CS094)

Rethin Francis (LTV22CS149)

ABSTRACT

Recent advancements in computer vision have enabled the reconstruction of three-dimensional structures from ordinary two-dimensional images. However, most existing solutions depend on cloud servers or high-performance computing systems, making them unsuitable for resource-constrained and offline environments.

This project proposes a fully offline 2D-to-3D reconstruction system optimized for ARM-based mobile devices. The system combines lightweight neural networks for depth estimation with efficient geometric processing techniques. It captures one or more images using a mobile camera, estimates depth maps using an on-device model, computes camera poses through feature matching, and fuses multiple depth maps into a unified 3D representation. A surface mesh is then generated and rendered in real time directly on the mobile device.

The proposed approach demonstrates that accurate and efficient 3D reconstruction can be achieved entirely on low-power mobile hardware through model optimization and efficient processing pipelines. This enables portable 3D scanning without the need for internet connectivity or external computational resources.

The system has applications in mobile 3D scanning, augmented reality, digital content creation, architectural measurement, and robotics. By eliminating cloud dependency, the solution ensures privacy, reduces latency, and improves accessibility. This work contributes toward the development of scalable, cost-effective, and fully offline 3D reconstruction systems for mobile and embedded platforms.

Contents

List of Figures	v
List of Tables	vi
Abbreviations	vii
Notation	vii
1 Introduction	1
2 Existing Methods	5
2.1 Cloud-Based Reconstruction Systems	6
2.2 3D Reconstruction Pipeline	7
2.3 Motivation	9
3 Problem Statement and Objectives	10
3.1 Problem Statement	11
3.2 Objectives	12
3.3 System Requirements	13
4 Design and Implementation	15
4.1 System Architecture	15
4.2 Implementation Details	16
5 Results and Discussion	17
5.1 Introduction	17
5.2 ARCore-Based Data Acquisition and Image Capture	17

5.2.1	Spatial Anchoring and Tracking	18
5.2.2	Camera Position and Angle of Capture	18
5.3	Depth Estimation via TensorFlow Lite MiDaS	21
5.3.1	Image Processing and Inference	21
5.4	Point Cloud Generation	23
5.4.1	Back-Projection Implementation	23
5.5	Surface Reconstruction and Mesh Rendering	25
5.5.1	The Rolling Ball Algorithm	25
5.5.2	Device Rendering and Asset Export	25
6	Conclusion and Future Scope	29
	References	30

List of Figures

2.1	Block diagram of the device	8
5.1	The AR capture interface showing the 1:1 aspect ratio square and spatial anchor.	19
5.2	Output data format illustrating the stored RGB image path, paired depth map, and the 4×4 camera pose matrix.	20
5.3	Application screenshot displaying the original captured RGB image (left) alongside its generated grayscale depth map (right).	22
5.4	Interactive 3D point cloud rendered within the mobile viewer, built from estimated depth data and mapped RGB colors.	24
5.5	The fully generated 3D mesh displayed within the mobile application's native rendering engine.	27
5.6	The exported .obj, .mtl, and .png texture files loaded into a third-party 3D viewer, demonstrating cross-platform portability.	28

List of Tables

ABBREVIATIONS

(List in the alphabetical order)

NOTATION

(List in the alphabetical order)

Chapter 1

Introduction

Three-dimensional (3D) reconstruction from two-dimensional (2D) images has become a significant area of research in computer vision due to its wide range of practical applications. Technologies such as augmented reality, virtual reality, robotics, and digital modeling increasingly depend on accurate 3D representations of real-world objects. With the growing availability of smartphones, there is a strong interest in making such capabilities accessible directly on mobile devices.

Despite these advancements, most existing 3D reconstruction systems rely on cloud-based processing or high-performance computing resources. These approaches require continuous internet connectivity and powerful hardware, which limits their usability in real-world scenarios, especially in mobile and resource-constrained environments. Moreover, transmitting image data to external servers introduces concerns related to privacy, latency, and operational cost.

To overcome these limitations, this project focuses on developing a fully offline 2D-to-3D reconstruction system specifically designed for ARM-based mobile devices. The system aims to enable users to capture images using a smartphone and generate a 3D model directly on the device without any dependency on cloud services. This

makes the solution more portable, efficient, and privacy-preserving.

The proposed system integrates lightweight deep learning models with efficient geometric processing techniques. It captures one or more images using the mobile camera, estimates depth information through an on-device neural network, determines camera poses using feature matching, and combines multiple depth maps into a unified 3D representation. The final output is a reconstructed 3D model that can be visualized in real time on the mobile device.

By eliminating the need for external computation, the system reduces latency and improves reliability in offline conditions. This approach opens up new possibilities for applications such as mobile 3D scanning, architectural visualization, digital content creation, and offline augmented reality experiences. Overall, the project demonstrates that efficient and practical 3D reconstruction can be achieved on low-power mobile hardware through careful system design and optimization.

In recent years, the field of computer vision has witnessed rapid progress due to the integration of deep learning techniques. Neural networks have significantly improved the ability to extract meaningful information from images, enabling tasks such as object detection, segmentation, and depth estimation. Among these, depth estimation plays a crucial role in understanding the spatial structure of a scene, which is essential for 3D reconstruction.

However, achieving accurate depth estimation on mobile devices remains a challenging task. Mobile processors have limited computational power and memory compared to desktop GPUs. Therefore, models must be carefully optimized to ensure efficient execution without compromising too much on accuracy. Techniques such as model quantization, pruning, and the use of lightweight architectures are commonly employed to address these challenges.

Another important aspect of 3D reconstruction is camera pose estimation. Accurately determining the position and orientation of the camera for each captured frame is essential for aligning multiple images into a consistent 3D structure. Feature-based methods are commonly used for this purpose, where distinctive points in images are detected and matched across frames. These correspondences are then used to estimate the relative motion between frames.

Depth fusion is the process of combining multiple depth maps into a single coherent representation. This step is critical in reducing noise and improving the overall quality of the reconstructed model. Efficient fusion techniques are required to ensure that the system can operate in real time on mobile devices. Following fusion, surface reconstruction algorithms are used to generate a mesh that represents the geometry of the scene.

The importance of mobile-based 3D reconstruction extends beyond academic research. In practical scenarios, users often require quick and portable solutions that do not depend on external infrastructure. For example, in construction and architecture, on-site 3D scanning can help in measuring spaces and visualizing designs. In healthcare, 3D models can assist in diagnostics and treatment planning. Similarly, in education and entertainment, interactive 3D content enhances user engagement.

Privacy is another critical factor that motivates the development of offline systems. Many existing solutions require uploading images to cloud servers, which may expose sensitive information. By performing all computations locally on the device, the proposed system ensures that user data remains secure and under the user's control.

Furthermore, the advancement of mobile hardware, including improved CPUs, GPUs, and dedicated AI accelerators, has made it increasingly feasible to run complex algorithms directly on smartphones. This trend supports the development of applications that were previously limited to high-performance computing environments.

In addition to technical challenges, usability is also an important consideration.

The system should provide an intuitive interface that allows users to easily capture images and visualize results. Efficient memory management and optimized processing pipelines are necessary to ensure smooth performance without excessive battery consumption.

The proposed project aims to bridge the gap between high-quality reconstruction systems and the limitations of mobile devices. By carefully combining modern machine learning techniques with efficient algorithm design, it is possible to achieve a balance between accuracy, speed, and resource usage.

In conclusion, this project contributes to the growing field of mobile computer vision by demonstrating that 3D reconstruction can be performed effectively in a fully offline environment. It highlights the potential of mobile devices as powerful tools for real-world applications and paves the way for future advancements in portable 3D scanning technologies.

Chapter 2

Existing Methods

Three-dimensional reconstruction from two-dimensional images has been an active area of research in computer vision for many years. Several techniques have been developed to generate 3D structures from image data, each with its own advantages and limitations. Most existing methods focus on achieving high accuracy and detailed reconstruction, often relying on computationally intensive algorithms or deep learning models.

Traditional approaches such as Structure from Motion (SfM) and Multi-View Stereo (MVS) use multiple images captured from different viewpoints to estimate camera positions and reconstruct the 3D structure of a scene. These methods are widely used due to their accuracy, but they require significant processing power and are not suitable for real-time or mobile-based applications.

Recent advancements in deep learning have introduced monocular depth estimation models that can predict depth from a single image. Models such as MiDaS and ZoeDepth have shown promising results in generating dense depth maps. However, these models are typically large and require optimization before they can be deployed efficiently on mobile devices.

In addition to these methods, sensor-based approaches using depth cameras such as LiDAR and structured light have also been explored. These systems provide accurate depth information but require specialized hardware, which increases the cost and limits accessibility for general users. As a result, purely vision-based methods remain an attractive solution for mobile applications.

2.1 CLOUD-BASED RECONSTRUCTION SYSTEMS

Another widely used approach is cloud-based 3D reconstruction. In this method, images captured from a device are uploaded to remote servers where powerful GPUs process the data and generate a 3D model. This approach produces high-quality results and supports complex reconstruction tasks.

However, cloud-based solutions introduce several challenges. They depend on stable internet connectivity, which may not always be available. Additionally, sending image data to external servers raises privacy concerns and increases latency. These limitations make such systems less practical for real-time and offline applications.

Some mobile-based solutions have also been proposed to perform reconstruction directly on smartphones. These systems aim to reduce dependency on external resources but often compromise on accuracy or processing speed. Many of them do not fully utilize modern deep learning techniques, limiting their performance.

Another drawback of cloud-based systems is scalability. As the number of users increases, the demand for server resources also grows, leading to higher operational costs. Furthermore, large-scale data transfer can result in bandwidth limitations, especially in regions with poor network infrastructure. These issues further highlight

the need for efficient on-device solutions.

2.2 3D RECONSTRUCTION PIPELINE

A typical 3D reconstruction pipeline consists of multiple stages, including image acquisition, feature extraction, camera pose estimation, depth estimation, and mesh generation. Feature matching techniques are used to identify correspondences between images, which helps in estimating camera motion.

Depth maps obtained from sensors or neural networks are combined using fusion techniques to create a unified 3D representation. Finally, surface reconstruction algorithms generate a mesh that represents the object or scene. While this pipeline is effective, it is computationally expensive and difficult to execute efficiently on low-power mobile devices.

Each stage of the pipeline introduces its own set of challenges. For instance, feature extraction must be robust to variations in lighting and viewpoint. Camera pose estimation requires accurate matching of features across frames, which can be affected by noise or motion blur. Depth estimation models must balance accuracy and computational efficiency, especially when deployed on mobile hardware.

Moreover, mesh generation and visualization require efficient data structures and rendering techniques to ensure smooth performance. Without proper optimization, these processes can lead to high memory consumption and reduced frame rates, affecting the overall user experience.

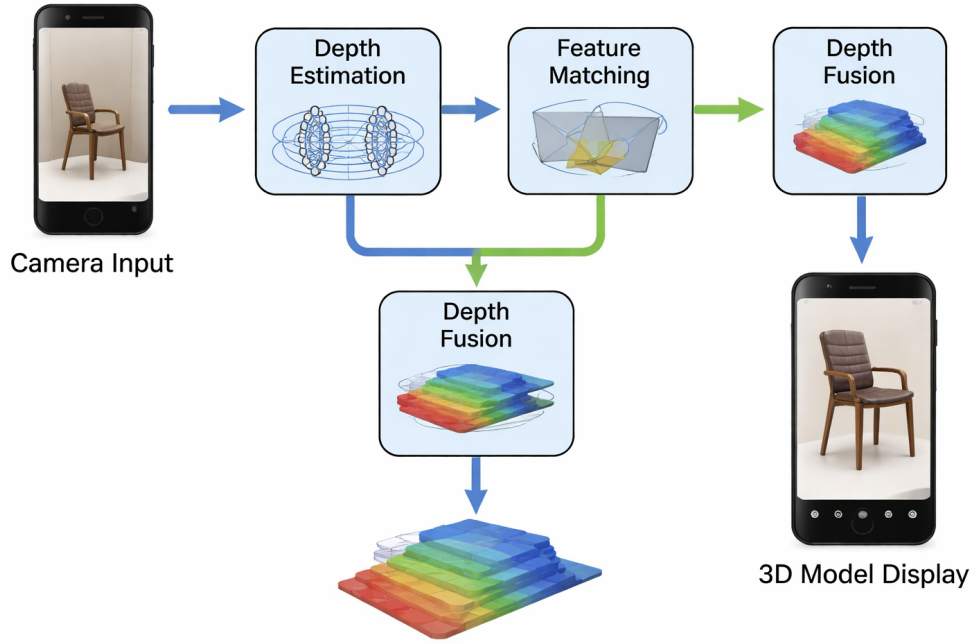


Figure 2.1: Block diagram of the device

From the analysis of existing methods, it is clear that there is a trade-off between accuracy and efficiency. High-quality reconstruction systems require powerful hardware or cloud support, while lightweight solutions struggle to maintain accuracy. This highlights the need for a balanced approach that can deliver acceptable performance while remaining efficient and portable.

Furthermore, the integration of deep learning with traditional geometric methods has opened new possibilities, but also introduced additional computational challenges. Optimizing these hybrid approaches for mobile environments remains an important area of research.

2.3 MOTIVATION

The limitations of existing systems highlight the need for a more practical solution that can operate entirely on mobile devices. With the increasing demand for applications such as augmented reality, mobile scanning, and offline visualization, it is important to develop systems that are both efficient and independent of internet connectivity.

This project is motivated by the goal of enabling 3D reconstruction directly on ARM-based mobile devices without relying on external computation. By combining lightweight deep learning models with optimized geometric processing techniques, the proposed system aims to achieve a balance between performance, accuracy, and usability in real-world scenarios.

In addition, the growing availability of smartphones with improved processing capabilities provides an opportunity to shift complex computations from cloud environments to edge devices. This transition not only enhances privacy but also reduces dependency on external infrastructure.

The motivation also stems from the need to make 3D reconstruction technology more accessible to a wider audience. By eliminating the requirement for specialized hardware or high-end systems, the proposed solution enables everyday users to generate 3D content easily using their mobile devices.

Finally, this work aims to contribute toward the development of efficient and scalable mobile vision systems, paving the way for future innovations in areas such as real-time augmented reality, digital twin creation, and intelligent mobile sensing.

Chapter 3

Problem Statement and Objectives

With the increasing demand for 3D content in applications such as augmented reality, robotics, and digital modeling, there is a growing need for efficient methods to reconstruct 3D structures from 2D images. While many solutions exist, most of them are designed for high-performance systems and are not suitable for mobile or offline environments.

Mobile devices, especially those based on ARM architectures, have limited computational power compared to desktops or cloud servers. Therefore, developing a system that can perform accurate 3D reconstruction directly on such devices presents several challenges. This chapter outlines the key problem addressed in this project and the objectives defined to solve it.

In addition, the rapid growth of mobile applications has created a demand for intelligent systems that can process visual data locally. Users increasingly expect applications to function seamlessly without relying on network connectivity. This shift towards edge computing highlights the importance of designing algorithms that are both efficient and capable of running on resource-constrained devices.

3.1 PROBLEM STATEMENT

Most existing 2D-to-3D reconstruction systems depend on cloud computing or high-end hardware to achieve acceptable levels of accuracy and performance. These systems require continuous internet connectivity and access to powerful GPUs, which limits their usability in real-world scenarios.

Such approaches introduce several issues. First, they are not suitable for offline environments where internet access is unavailable or unreliable. Second, transmitting image data to external servers raises privacy and security concerns. Third, the dependency on cloud infrastructure increases latency and operational cost, making real-time applications difficult to achieve.

Additionally, traditional reconstruction techniques are computationally intensive and not optimized for resource-constrained devices like smartphones. As a result, there is a lack of efficient solutions that can perform 3D reconstruction directly on mobile devices while maintaining a balance between accuracy and performance.

Another important challenge is energy consumption. Continuous processing of images and running machine learning models can significantly drain battery power, making it impractical for long-term usage on mobile devices. Moreover, memory limitations restrict the size of models and data that can be processed on-device.

Therefore, the core problem addressed in this project is to design and implement a lightweight, efficient, and fully offline 2D-to-3D reconstruction system that can operate effectively on ARM-based mobile devices.

3.2 OBJECTIVES

The primary objective of this project is to develop a mobile-based system capable of reconstructing 3D models from 2D images without relying on external computation. To achieve this, the following specific objectives are defined:

- To design a system that captures one or more images using a mobile device camera.
- To implement a lightweight depth estimation module using on-device neural networks.
- To estimate camera poses through feature matching techniques between captured frames.
- To develop an efficient depth fusion method to combine multiple depth maps into a unified 3D representation.
- To generate and visualize a 3D surface model directly on the mobile device in real time.
- To ensure the system operates fully offline, preserving user privacy and reducing dependency on cloud services.

In addition to these core objectives, the project also aims to ensure that the system is user-friendly and easy to operate. The interface should allow users to capture images, process data, and visualize results without requiring technical expertise. Furthermore, the system should maintain a balance between computational efficiency and reconstruction quality.

These objectives collectively aim to create a practical and efficient solution for mobile 3D reconstruction.

3.3 SYSTEM REQUIREMENTS

In addition to the primary objectives, the project also focuses on optimizing performance and usability. This includes reducing computational overhead, minimizing memory usage, and ensuring smooth user interaction during the capture and visualization process.

Another important consideration is scalability. The system should be adaptable to different mobile devices with varying hardware capabilities. Furthermore, the design should allow future improvements, such as integrating advanced depth estimation models or enhancing reconstruction accuracy.

The system must also satisfy certain functional and non-functional requirements. Functional requirements include capturing images, processing depth information, generating 3D models, and displaying results to the user. Non-functional requirements include efficiency, reliability, low latency, and minimal resource consumption.

In terms of software requirements, the system should support mobile platforms and integrate frameworks such as ARCore and TensorFlow Lite for efficient implementation. Hardware requirements include a smartphone with camera capabilities and sufficient processing power to handle real-time operations.

By addressing these aspects, the project aims not only to solve the core problem but also to provide a flexible and user-friendly solution suitable for real-world applications.

In summary, this chapter defines the challenges associated with existing 3D reconstruction methods and outlines the objectives of the proposed system. The focus is on developing a lightweight, efficient, and fully offline solution that bridges the gap between high-performance reconstruction techniques and the limitations of mobile devices.

The identified problem and defined objectives serve as the foundation for the system design and implementation discussed in the following chapters.

Chapter 4

Design and Implementation

This chapter presents the overall design and implementation details of the proposed offline 2D-to-3D reconstruction system. The system is designed to operate efficiently on ARM-based mobile devices by integrating lightweight machine learning models with optimized geometric processing techniques. A modular architecture is adopted to ensure scalability, maintainability, and ease of development.

The implementation focuses on capturing image data, processing it through multiple stages, and generating a 3D model directly on the mobile device. Each component of the system is carefully designed to balance performance and accuracy while operating under the constraints of limited computational resources.

4.1 SYSTEM ARCHITECTURE

The system follows a modular architecture where each major functionality is implemented as an independent module. The key modules of the system include image capture, depth estimation, 3D reconstruction, and visualization.

The image capture module utilizes the mobile device camera and AR capabilities to acquire RGB images along with depth information and camera pose. The depth

estimation module enhances the captured data using lightweight neural networks deployed through TensorFlow Lite. The reconstruction module processes the depth maps and camera poses to generate a unified 3D representation. Finally, the visualization module renders the reconstructed model for user interaction.

This modular design allows individual components to be developed and tested independently while ensuring smooth integration into the complete system.

4.2 IMPLEMENTATION DETAILS

The system is implemented as a mobile application using Flutter for the user interface and native Android components for AR functionalities. ARCore is used to capture real-time camera data, including depth information and pose estimation. The captured data is stored locally on the device in an organized format for further processing.

Depth estimation is performed using optimized machine learning models converted to TensorFlow Lite format, enabling efficient on-device inference. The reconstruction process involves aligning depth maps using camera pose information, followed by depth fusion to generate a consistent 3D structure.

The final stage involves mesh generation and visualization. Point cloud data is processed to create a surface mesh, which is then rendered using lightweight 3D visualization libraries. The application also includes features for managing captured sessions and exporting data for further use.

Overall, the design and implementation focus on achieving a balance between efficiency and functionality. By combining mobile-friendly technologies with optimized algorithms, the system successfully demonstrates the feasibility of performing 3D reconstruction entirely on a mobile device without external computation.

Chapter 5

Results and Discussion

5.1 INTRODUCTION

This chapter presents the practical outcomes, implementation strategies, and key discussion points of the 2D-to-3D Image Conversion project. The pipeline consists of four major operational phases: Augmented Reality (AR)-based image acquisition, neural network-driven depth estimation, point cloud back-projection, and geometric surface reconstruction via the rolling ball algorithm. The following sections provide an in-depth analysis of each module's implementation and performance on a mobile edge device.

5.2 ARCORE-BASED DATA ACQUISITION AND IMAGE CAPTURE

The initial and most crucial step of the pipeline is the acquisition of high-quality, geometrically contextualized images. This was implemented using Google's ARCore framework integrated into the Flutter application.

5.2.1 Spatial Anchoring and Tracking

Instead of relying on a simple smartphone camera capture, the application initializes an AR session. This allows the system to perceive the physical environment while tracking feature points and planar surfaces. The user is prompted to place a virtual **Anchor** in the real world. This anchor acts as the absolute origin (0,0,0) for the coordinate system of that specific scanning session. By localizing the capture around this anchor, the system achieves a grounded reference frame.

5.2.2 Camera Position and Angle of Capture

When the user triggers the capture mechanism, the system records not only RGB pixel data but also spatial metadata. The exact **camera position and orientation** (6 Degrees of Freedom - 6DoF) are recorded as a 4×4 relative pose matrix representing the camera's spatial relationship to the placed anchor. This specific angle of capture is vital; it ensures that, if the system is scaled to support multi-view stereo in the future, the transformation constraints between different viewing angles are preserved. To prevent spatial distortion during the neural network inference stage, the capture module enforces a strict 1:1 (square) aspect ratio crop, guaranteeing that the true proportions of the localized object are preserved.

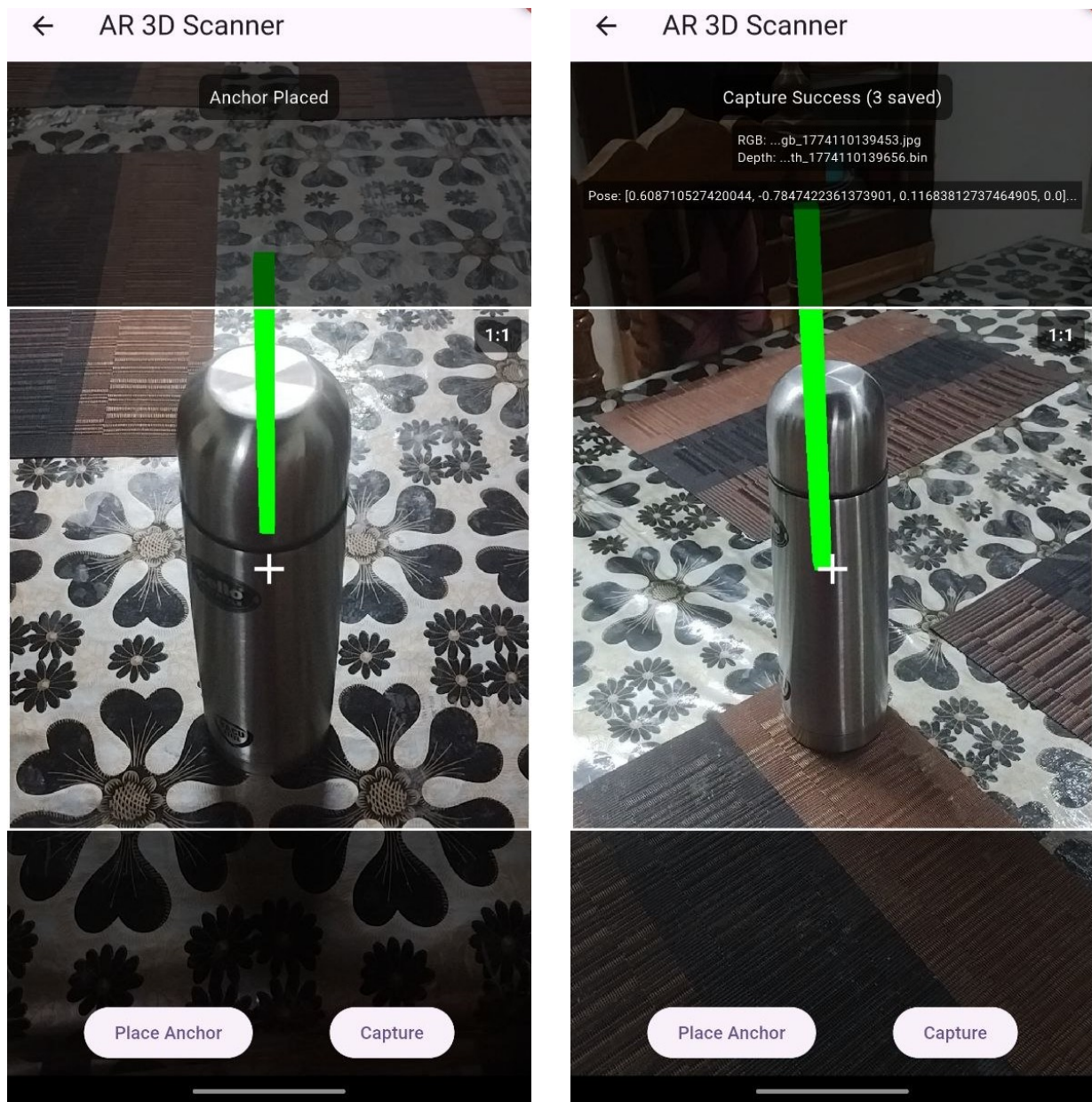


Figure 5.1: The AR capture interface showing the 1:1 aspect ratio square and spatial anchor.

```

{} captures.json X
visualizations > session_for_report > {} captures.json > ...
1  [
2  {
3    "imagePath": "/data/user/0/com.example.app/app_flutter/captures/session_1774110049571/rgb_1774110116046.jpg",
4    "depthPath": "/data/user/0/com.example.app/app_flutter/captures/session_1774110049571/depth_1774110116262.bin",
5    "relativePose": [
6      0.27592986822128296, -0.8592849969863892, 0.4306878447532654, 0.0,
7      0.831463098526001, -0.011411428451538086, -0.5554628968238831, 0.0,
8      0.48221564292907715, 0.5113698840141296, 0.7113149166107178, 0.0,
9      0.24309834837913513, 0.2779819965362549, 0.33068937063217163, 1.0
10   ],
11   "timestamp": "2026-03-21T21:51:56.290299"
12 },
13 {
14   "imagePath": "/data/user/0/com.example.app/app_flutter/captures/session_1774110049571/rgb_1774110129319.jpg",
15   "depthPath": "/data/user/0/com.example.app/app_flutter/captures/session_1774110049571/depth_1774110129450.bin",
16   "relativePose": [
17     -0.26665425300598145, -0.7793031930923462, 0.5670821070671082, 0.0,
18     0.9036002159118652, 0.0025300979614257812, 0.4283693730831146, 0.0,
19     -0.3352643847465515, 0.6266419887542725, 0.7035038471221924, 0.0,
20     -0.14016880095005035, 0.28429165482521057, 0.3291293978691101, 1.0
21   ],
22   "timestamp": "2026-03-21T21:52:09.457531"
23 },
24 {
25   "imagePath": "/data/user/0/com.example.app/app_flutter/captures/session_1774110049571/rgb_1774110139453.jpg",
26   "depthPath": "/data/user/0/com.example.app/app_flutter/captures/session_1774110049571/depth_1774110139656.bin",
27   "relativePose": [
28     0.608710527420044, -0.7847422361373901, 0.11683812737464905, 0.0,
29     0.14896631240844727, -0.031600356101989746, -0.9883372187614441, 0.0,
30     0.7792820930480957, 0.6190162301063538, 0.0976647138595581, 0.0,
31     0.3810936212539673, 0.29566067457199097, 0.056495167315006256, 1.0
32   ],
33   "timestamp": "2026-03-21T21:52:19.657403"
34 }
35 ]

```

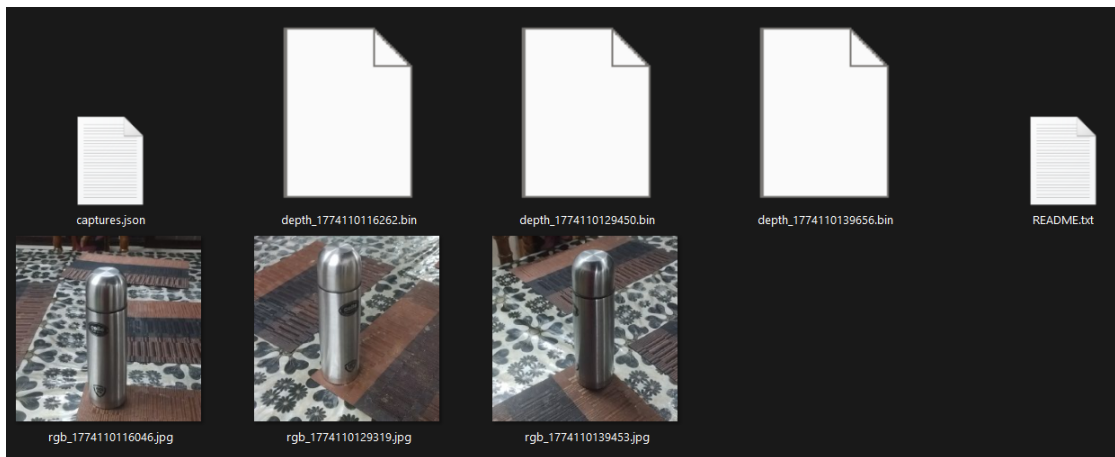


Figure 5.2: Output data format illustrating the stored RGB image path, paired depth map, and the 4×4 camera pose matrix.

5.3 DEPTH ESTIMATION VIA TENSORFLOW LITE MiDaS

Once the 1:1 RGB image is captured, it is processed locally using an on-device TensorFlow Lite (TFLite) interpreter running the MiDaS (Small) architecture.

5.3.1 Image Processing and Inference

The captured high-resolution square image is downsampled to a strict 256×256 pixel resolution, matching the input tensor shape expected by the MiDaS model. Pixel integers (0–255) are converted into 32-bit floating-point arrays and normalized between 0.0 and 1.0 across the RGB channels.

The inference engine executes the model, which outputs an inverse relative depth map. A significant implementation hurdle was observed: the raw output yielded inverted depth (making distant objects appear erroneously close). This was resolved programmatically by applying an inverse scale mapping formula: $Z = 0.5 + (D_{norm} \times 2.5)$, where D_{norm} is the normalized depth value. This effectively bounds the predicted spatial range between 0.5 meters (foreground) and 3.0 meters (background).

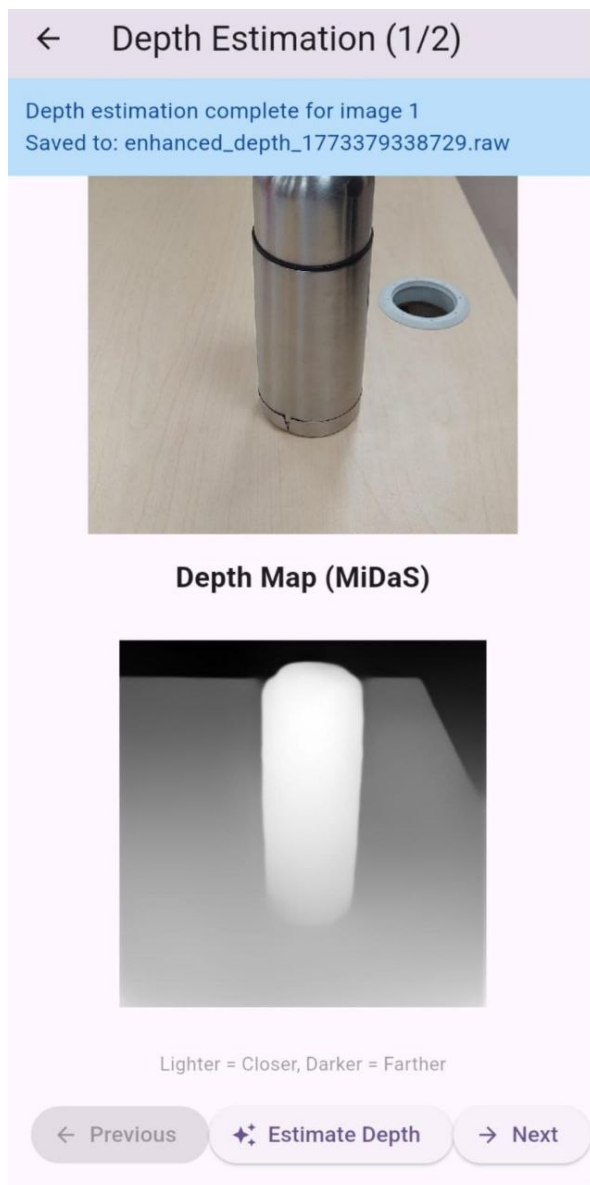


Figure 5.3: Application screenshot displaying the original captured RGB image (left) alongside its generated grayscale depth map (right).

5.4 POINT CLOUD GENERATION

The transformation from 2D arrays to a 3D coordinate space is achieved via intrinsic camera back-projection.

5.4.1 Back-Projection Implementation

Using a pinhole camera model, the algorithm iterates through every pixel $p(u, v)$ in the resultant depth map. Given the estimated depth Z at that pixel, and the estimated horizontal/vertical focal lengths (f_x, f_y) combined with the optical center (c_x, c_y) , the absolute 3D coordinates (X, Y, Z) are reconstructed. Each coordinate is simultaneously mapped back to the original high-resolution RGB image to extract the exact R, G, B color values, embedding the visual texture directly into the geometric points. Stride factors were implemented to allow dynamic downsampling, ensuring the mobile device's memory limits are not exceeded while maintaining dense environmental structures.

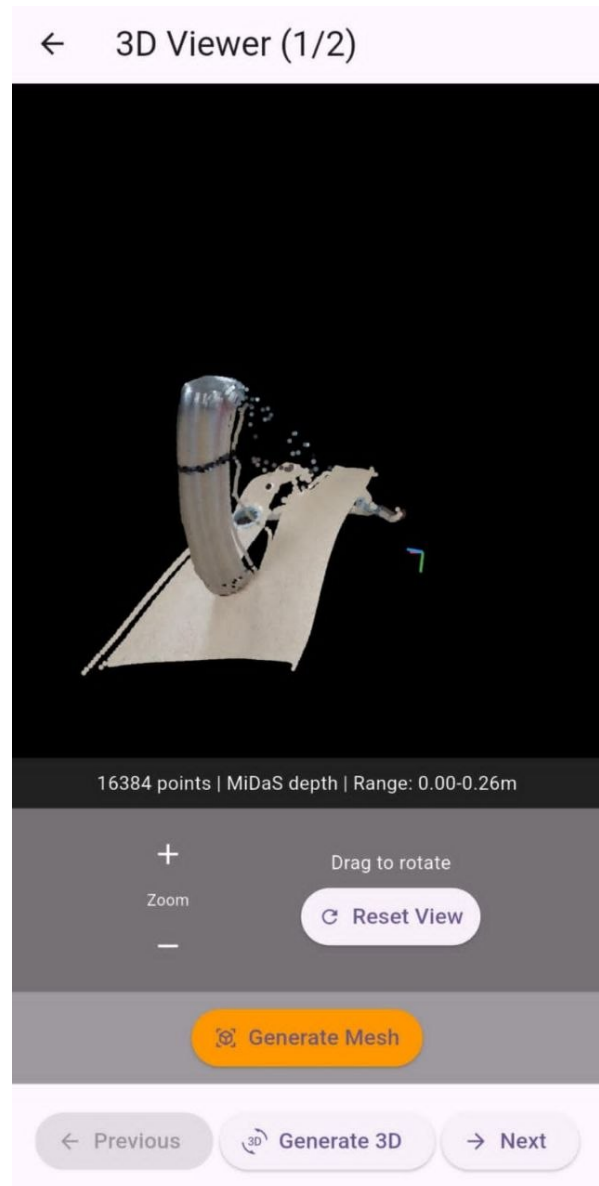


Figure 5.4: Interactive 3D point cloud rendered within the mobile viewer, built from estimated depth data and mapped RGB colors.

5.5 SURFACE RECONSTRUCTION AND MESH RENDERING

The final phase involves deriving a continuous topological surface from the discrete point cloud, enabling the object's use in standard 3D rendering engines.

5.5.1 The Rolling Ball Algorithm

To create valid triangular faces connecting the points, the **Ball Pivoting Algorithm (Rolling Ball Algorithm)** was implemented. The algorithm simulates a virtual ball of a defined radius (e.g., $r = 0.05$ units) resting on three points in the cloud, forming a seed triangle. The ball then rolls over the edge of this triangle until it catches a new adjacent point without encapsulating any other points inside its volume, thereby forming a new adjacent triangle.

To ensure the mesh is structurally sound and to prevent gaping holes across varying point densities, several heuristic constraints were added:

- **Maximum Edge Length:** Triangles are rejected if the distance between vertices exceeds a threshold, preventing stretched, artificial geometry between completely separate objects (e.g., an object and the wall behind it).
- **Neighborhood Search Constraints:** Optimization was performed using a localized voxel grid, which drastically reduced computation time by testing the virtual ball only against nearby points rather than iterating over the entire cloud.

5.5.2 Device Rendering and Asset Export

The generated mesh layout is rendered natively on the device using the `flutter_cube` parsing library. Vertex color shading is used to render the object through its assigned texture data, overriding artificial lighting shaders to preserve the original appearance.

Furthermore, an exporter module serializes these data structures into widely supported graphics formats. The system exports a Wavefront `.obj` file containing

geometric vertices (v) and polygonal faces (f). For textures, it generates a matching Material Template Library (.mtl) file and a .png image. The UV map is computed to align mesh triangles with their corresponding regions in the generated 2D texture, enabling cross-platform sharing and analysis in standard 3D software.

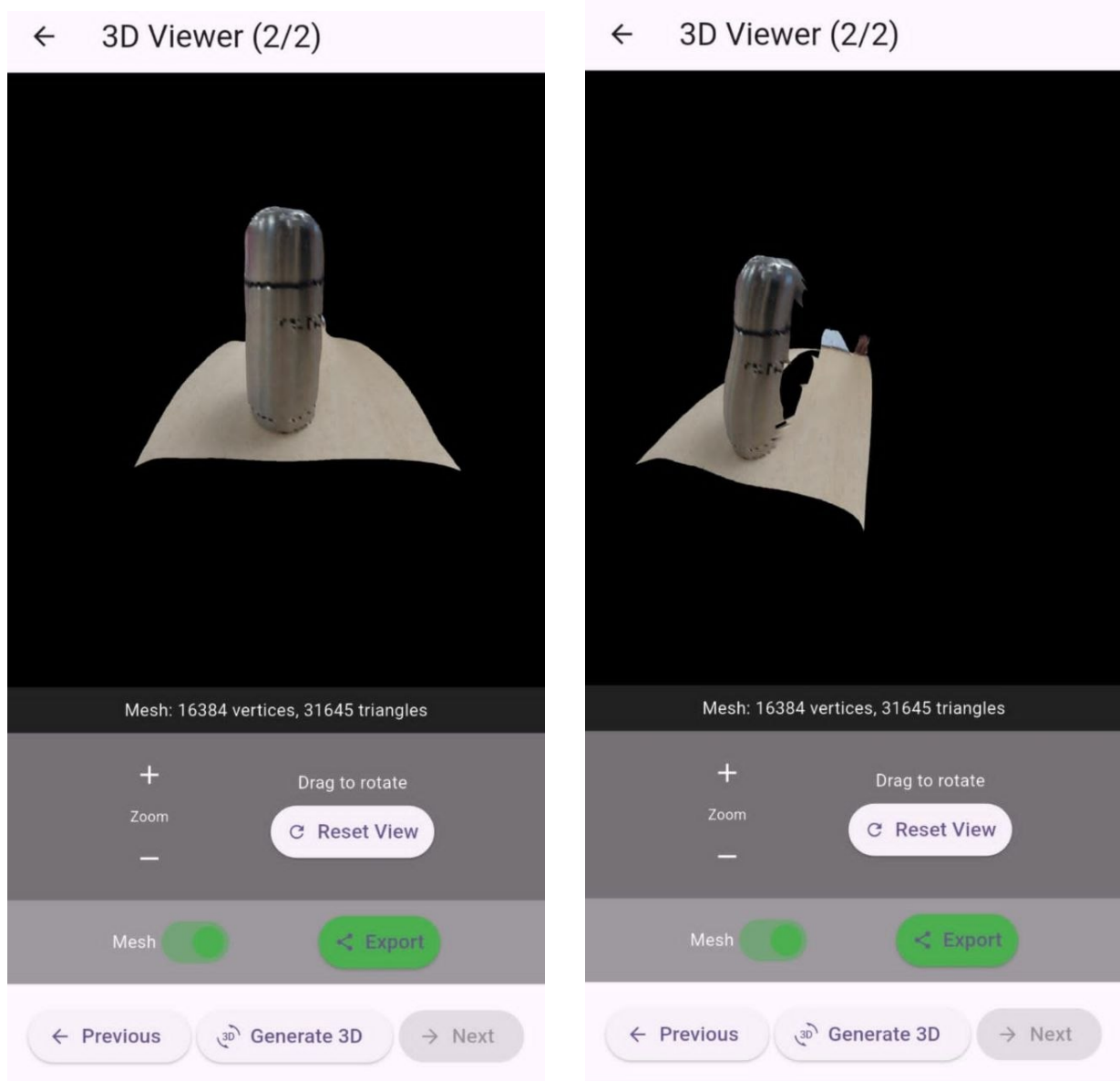


Figure 5.5: The fully generated 3D mesh displayed within the mobile application's native rendering engine.

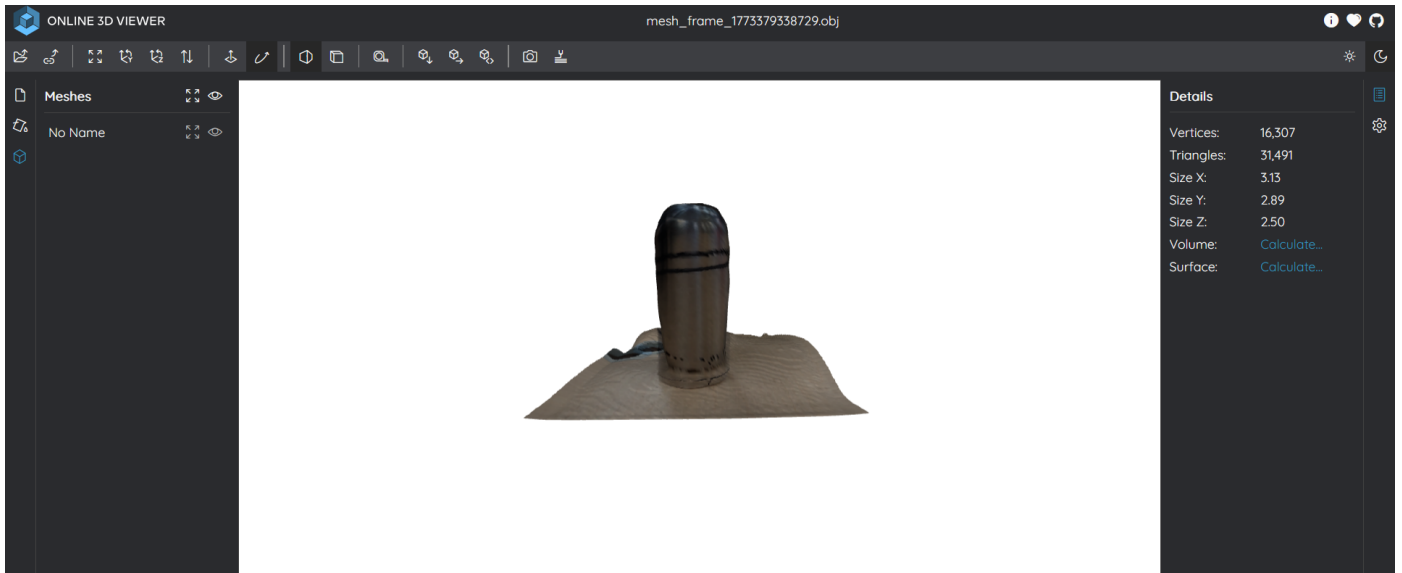
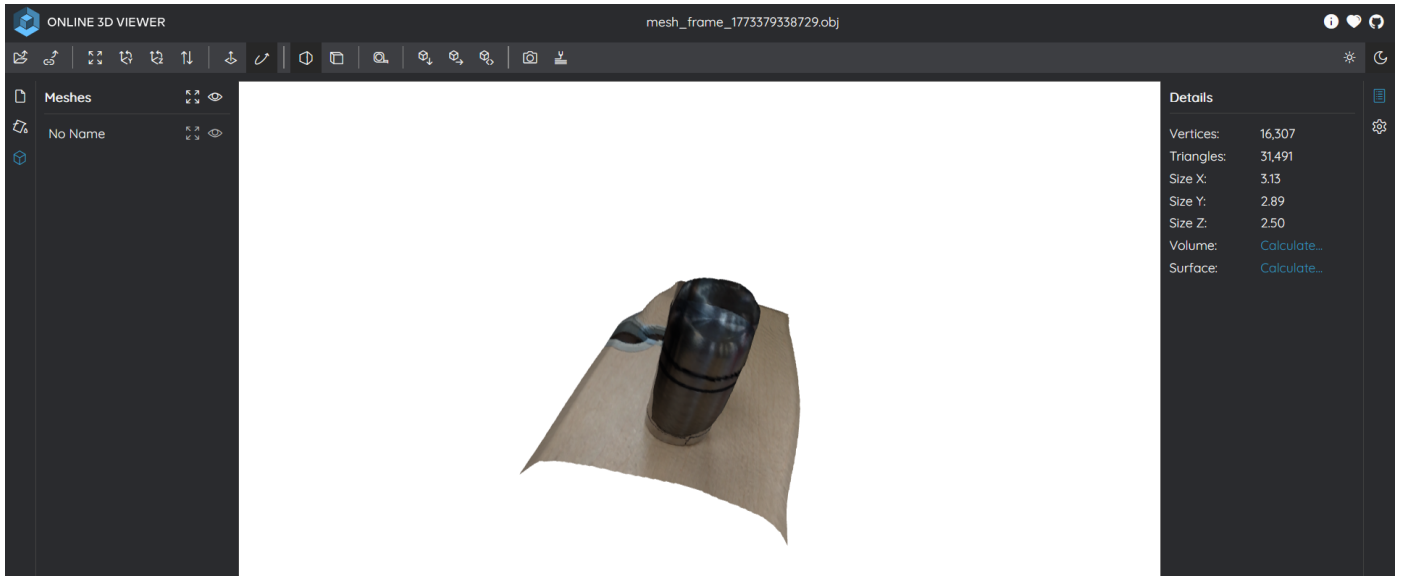


Figure 5.6: The exported .obj, .mtl, and .png texture files loaded into a third-party 3D viewer, demonstrating cross-platform portability.

Chapter 6

Conclusion and Future Scope

Bacon ipsum dolor amet shoulder cow shankle kevin, ham picanha strip steak biltong short loin tri-tip. Sausage burgdoggen pastrami, pork belly doner strip steak prosciutto salami spare ribs sirloin pork shankle. Rump beef ribs kevin cupim short loin tenderloin. Drumstick kevin pork spare ribs pork chop ground round. Picanha kevin prosciutto tail, kielbasa tri-tip ham landjaeger hamburger ground round short ribs.

Boudin rump beef ribs turkey pork chop meatball tri-tip andouille chuck brisket pork loin pig ball tip. Turkey strip steak ball tip pork salami. Shoulder salami ham hock strip steak meatloaf. Strip steak tail tri-tip chicken. Hamburger ribeye kielbasa, brisket flank capicola short loin turkey tail chuck bacon.

Landjaeger sirloin short loin, t-bone tail boudin burgdoggen meatball ham cupim strip steak pig andouille. Short ribs biltong corned beef drumstick, tri-tip hamburger prosciutto ball tip beef spare ribs. Pork loin pastrami tri-tip, turkey frankfurter pork buffalo cow short ribs leberkas corned beef shoulder short loin. Pastrami ball tip sausage, brisket venison cow t-bone frankfurter. Beef landjaeger sirloin pork loin buffalo cow tri-tip brisket pork belly pork flank hamburger alcatra. Pig porchetta tenderloin kevin salami boudin kielbasa fatback.

References

- [1] Design and Implementation of a Tactile Stimulation Device to Increase Auditory Discrimination, *IEEE Transactions on Haptics* , (Volume: 10, Issue: 4, Oct.- Dec. 1 2017)
- [2] Haptic Technology: A comprehensive review on its applications and future prospects, *International Conference on Computing, Communication, Nanophotonics, Nanoscience, Nanomaterials and Nanotechnology*
- [3] R. H. Gault, "Progress in experiments on tactual interpretation of oral speech", *The Journal of Abnormal Psychology and Social Psychology*, vol. 19, pp. 155-159, 1924.
- [4] J. P. Das, "Stages of Reading Development," in Reading Difficulties and Dyslexia: An Interpretation for Teachers. *SAGE Publications India Pvt Ltd*, 2009.
- [5] Auditory Discrimination Test - <https://www.encyclopedia.com/medicine/encyclopedias-almanacs-transcripts-and-maps/auditory-discrimination-test>